VNUHCM, UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

SUBMISSION OF SELF-PRACTICE EVIDENCE FOR PROGRAMMING

PART 4

INSTRUCTORS: Bùi Duy Đăng
Nguyễn Thanh Tình

CLASS: 25C01

No.	Student ID	Full name
1	25127113	Võ Thiện Trí Nhân

Ho Chi Minh City - 12/2025

1 Question 1

Question 1

Given a string **s** that contains only alphabetic characters and spaces, complete the following C++ function to reverse the order of the words in the string.

```
string reverseWords(string s);
```

Input	Output
Hello World	World Hello
HCMUS is amazing	amazing is HCMUS

1.1 Algorithm

I use `<string>` because the question doesn't specify the range of characters.

1. Reverse the whole string: Hello World → dlroW olleH.
2. Reverse each word in the string: dlroW olleH → World Hello.
3. Return the string.

1.2 Code

```cpp
#include <iostream>
#include <string>
#include <cstring>
#include <cctype>

using namespace std;

// Function to check if the character is an alphabetic character
bool isAlpha(char c) {
    if ((('a' <= c) && (c <= 'z')) || (('A' <= c) && (c <= 'Z'))) {
        return true;
    }
    return false;
}

string reverseWords(string s) {
    // H e l l o   W o r l d
    // 0 1 2 3 4 5 6 7 8 9 10

    int num = s.length(); // Get the length of the string

    // Reverse the whole string
    // Stop when i and j meet in the middle
    for (int i = 0, j = num - 1; i < j; i++, j--) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
```

```
    }

    int start = 0, end = 0, numberofwords = 0;
    while (end < num) { // Stop when "end" reaches the end of the string
        while (end < num && isAlpha(s[end])) { // Count the characters in one word
            numberofwords++;
            end++;
        }

        // If we found a word, reverse it
        if (numberofwords > 0) {
            int wordEndIndex = end - 1;
            int wordStartIndex = start;

            // Reverse the word
            for (int k = wordStartIndex; k < (wordStartIndex + numberofwords / 2); k++) {
                int swapIndex = wordEndIndex - (k - wordStartIndex);
                char temp = s[k];
                s[k] = s[swapIndex];
                s[swapIndex] = temp;
            }
        }

        end++; // Skip the whitespace
        start = end;
        numberofwords = 0;
    }

    return s;
}

int main() {
    string s;
    getline(cin, s);
    cout << reverseWords(s) << endl;

    return 0;
}
```

1.3 Test cases

Test case 1: My name is Mike → Mike is name My.
Test case 2: I love FIT HCMUS → HCMUS FIT love I.
Test case 3: Treat me like white tee → tee white like me Treat.

2 Question 2

Question 2

Define a struct named `Circle` to represent a **circle** in the 2D Cartesian plane determined by:

- The coordinates of its center (x, y).
- Its radius r.

Then, complete the following function:

```
1 void printRelation(Circle c1, Circle c2);
```

This function prints the **geometric relationship** between two circles `c1` and `c2`.

Input	Output
$c_1(0,0,3)$, $c_2(0,0,3)$	Coincident
$c_1(0,0,3)$, $c_2(6,0,3)$	Externally tangent
$c_1(0,0,5)$, $c_2(3,0,2)$	Internally tangent
$c_1(0,0,5)$, $c_2(4,0,3)$	Intersect
$c_1(0,0,5)$, $c_2(1,1,1)$	One contains the other
$c_1(0,0,2)$, $c_2(10,0,2)$	Separate

2.1 Algorithm

To identify the relation between two circles, we need to calculate and compare the distance between the two centers with the sum or difference of the radii.

1. If the distance between two centers is equal to 0, they are coincident.

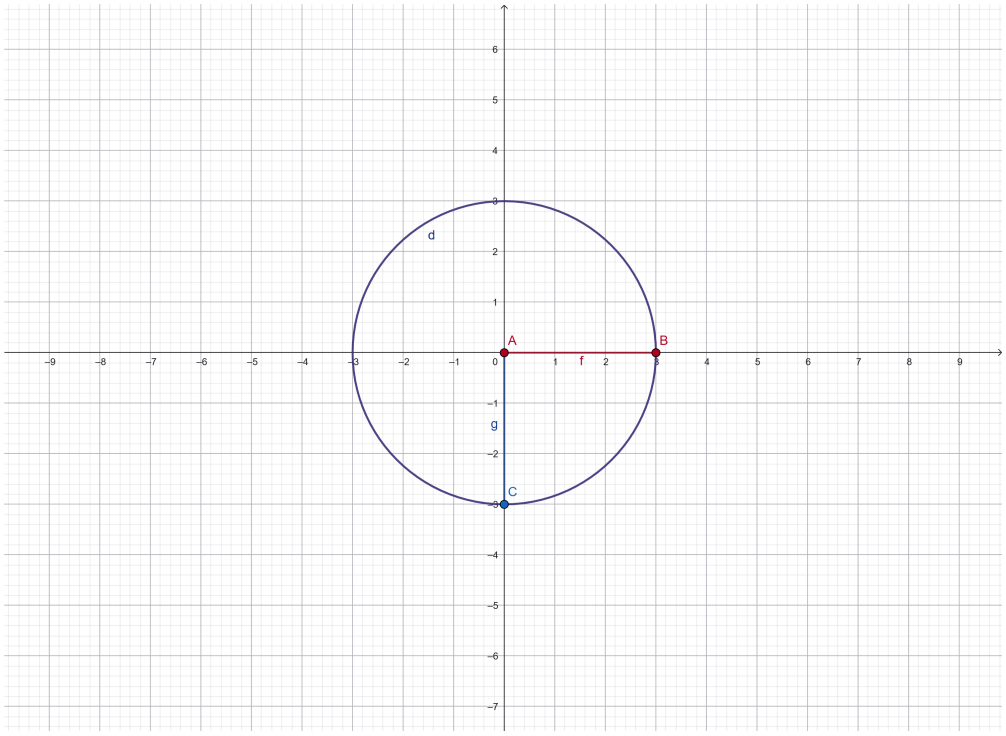

Figure 1: Visualization of two circles (0, 0, 3) and (0, 0, 3)

2. If the distance between two centers is equal to the sum of the two radii, they are externally tangent.

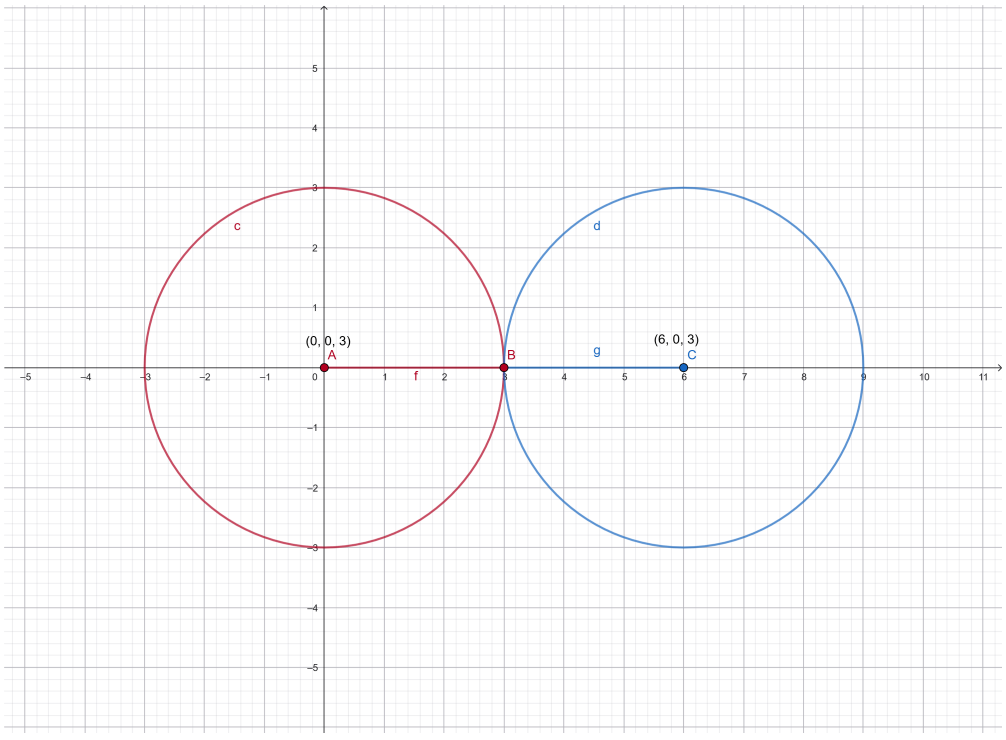

Figure 2: Visualization of two circles (0, 0, 3) and (6, 0, 3)

3. If the distance between two centers is equal to the difference of the two radii, they are internally

tangent.

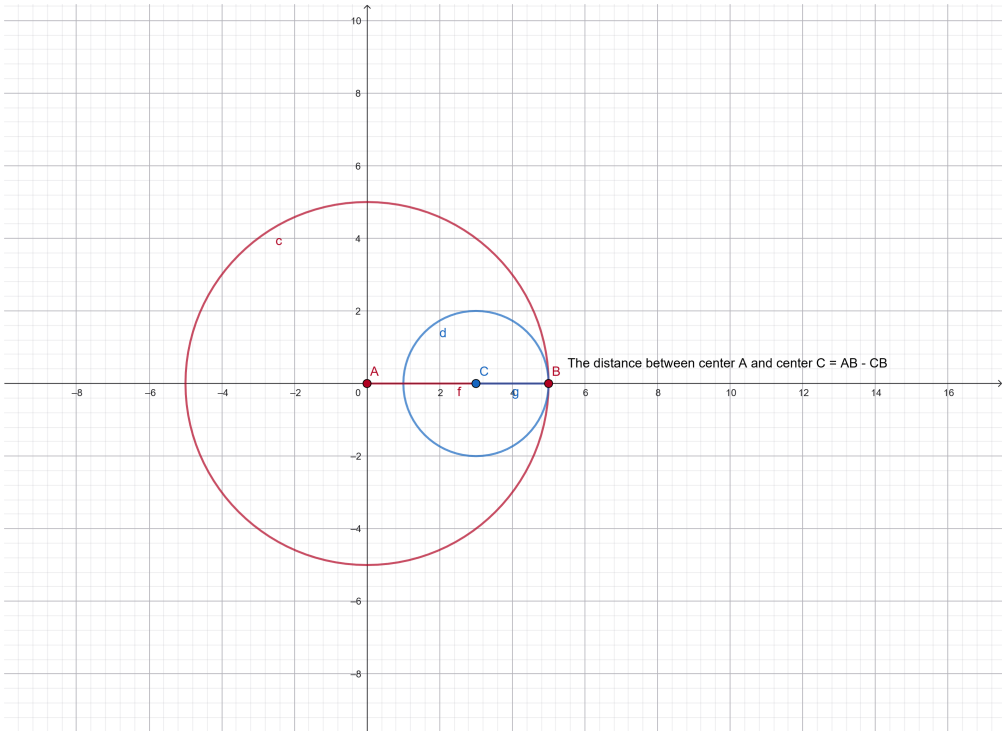

Figure 3: Visualization of two circles (0, 0, 5) and (3, 0, 2)

4. If the distance between two centers is less than the sum of the two radii (but greater than the difference), they intersect.

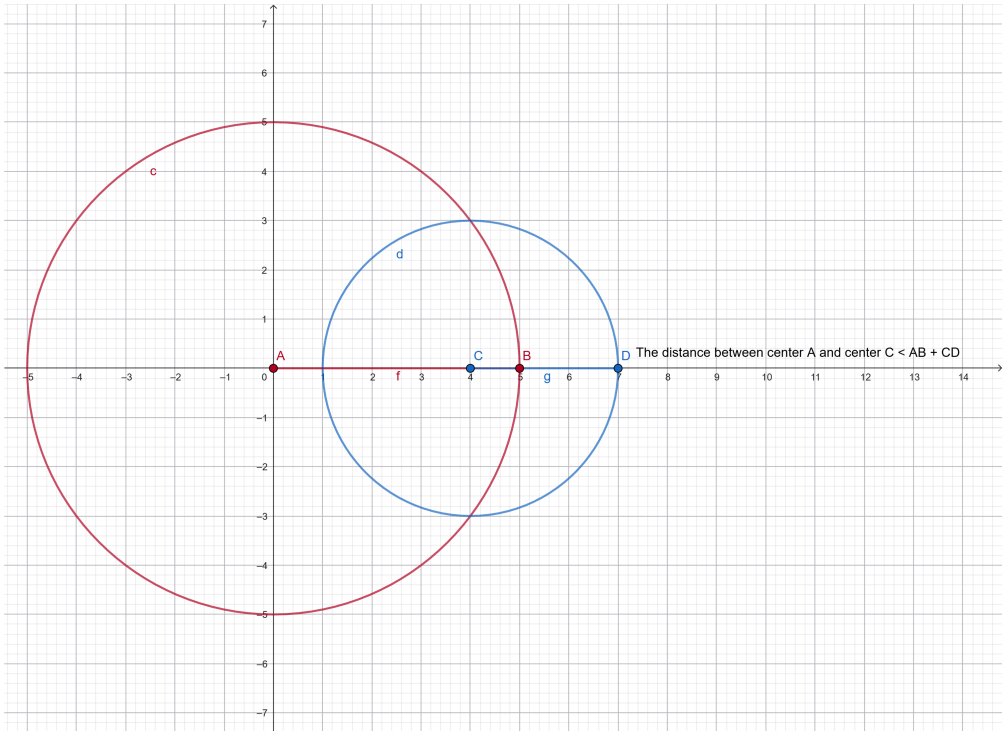

Figure 4: Visualization of two circles (0, 0, 5) and (4, 0, 3)

5. If the distance between two centers is less than the difference of the two radii, one circle is contained inside the other.

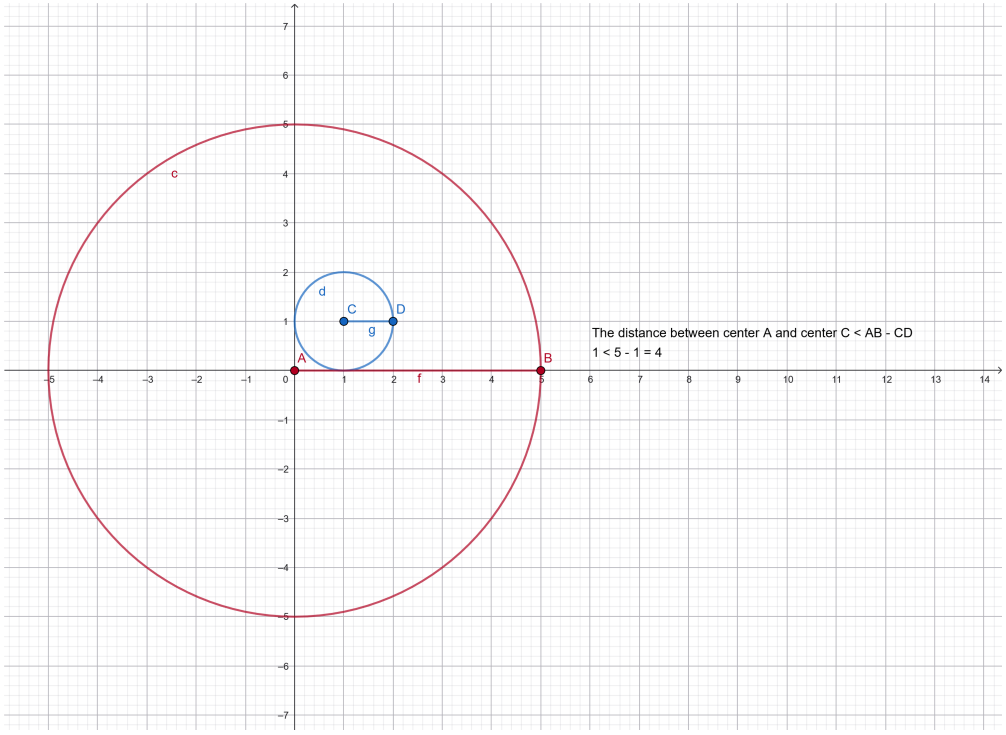

Figure 5: Visualization of two circles (0, 0, 5) and (1, 1, 1)

6. If the distance between two centers is greater than the sum of the two radii, they are separated.

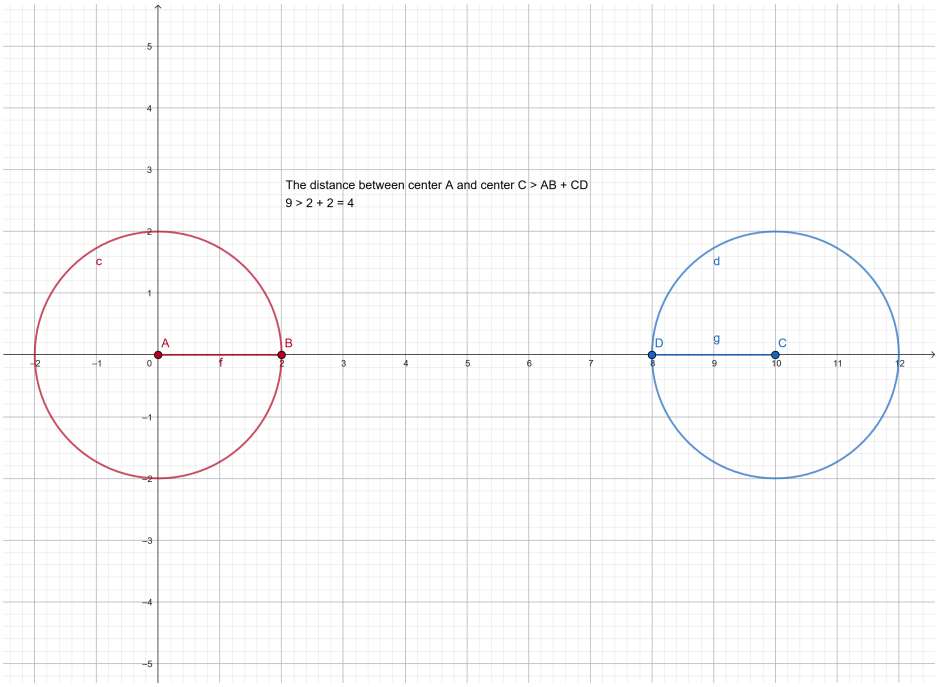

Figure 6: Visualization of two circles (0, 0, 2) and (10, 0, 2)

2.2 Code

```cpp
#include <iostream>
#include <cmath>

using namespace std;

// Structure holds circle properties
struct Circle {
    // The center of the circle
    float x;
    float y;
    // The radius of the circle
    float radius;
};

void printRelation(Circle c1, Circle c2) {
    // Find the distance between two centers
    // NOTE: Must use float/double to preserve decimal precision
    float distance_between_two_centers = sqrt(pow((c2.x - c1.x), 2) + pow((c2.y - c1.y), 2))
        ;

    // Note: Comparing floats with == is generally unsafe due to precision,
    // but kept here for simplicity as per algorithm description.
    if (distance_between_two_centers == 0) {
        cout << "Coincident" << endl;
    }
    else if (distance_between_two_centers == c1.radius + c2.radius) {
        cout << "Externally Tangent" << endl;
    }
    else if (distance_between_two_centers == abs(c1.radius - c2.radius)) {
        cout << "Internally Tangent" << endl;
```

```
    }
    else if (distance_between_two_centers < abs(c1.radius - c2.radius)) {
        cout << "One contains the other" << endl;
    }
    else if (distance_between_two_centers < (c1.radius + c2.radius)) {
        cout << "Intersect" << endl;
    }
    else if (distance_between_two_centers > (c1.radius + c2.radius)) {
        cout << "Separate" << endl;
    }
}

int main() {
    Circle c1, c2;
    cin >> c1.x >> c1.y >> c1.radius;
    cin >> c2.x >> c2.y >> c2.radius;

    printRelation(c1, c2);

    return 0;
}
```

2.3 Test cases

Test case 1: c1(3, 4, 5), c2(3, 4, 5) → Coincident.
Test case 2: c1(0, 0, 4), c2(10, 0, 3) → Separate.
Test case 3: c1(1, 1, 6), c2(3, 3, 2) → One contains the other.

3 Question 3

Question 3

A **magic square** is a square matrix of size $n \times n$ that satisfies the following conditions:

- The sums of all rows are equal.
- The sums of all columns are equal.
- The sums of the two main diagonals are equal.

You are given a **2D array using pointers** with the number of rows and columns.
Complete the following C++ function:

```
bool isMagicSquare(int** a, int nRows, int nCols);
```

The function returns:

- `true` if the matrix is a magic square.
- `false` otherwise.

For example
The matrix:

$$\begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$$

is a **magic square** because it satisfies the following conditions:

- Row sums:
$$8+1+6 = 3+5+7 = 4+9+2 = 15$$
- Column sums:
$$8+3+4 = 1+5+9 = 6+7+2 = 15$$
- Sums of the two main diagonals:
$$8+5+2 = 6+5+4 = 15$$

Therefore, the matrix is a magic square with magic constant 15.

3.1 Algorithm

I follow this algorithm:

1. Set a flag called `isMagic` to true (Assume the given 2D array is magic).
2. Start with the rows:
   (a) Calculate the sum of the first row to get a checking number.
   (b) If the sum of any other row is not equal to the checking number, set the flag false and return false.
3. Continue with the columns:
   (a) Calculate the sum of the first column to get a checking number.
   (b) If the sum of any other column is not equal to the checking number, set the flag false and return false.
4. Apply this algorithm to the diagonals.
5. If the flag remains true, then the matrix is magic.

3.2 Code

```cpp
#include <iostream>

using namespace std;

bool isMagicSquare(int** a, int nRows, int nCols) {
    if (nRows != nCols) return false; // Magic square must be square

    // Calculate the first row
    int target_sum = 0;
    for (int col = 0; col < nCols; col++) {
        target_sum += a[0][col];
    }

    // Check if other rows are equal
    for (int row = 1; row < nRows; row++) {
        int sum_cur_row = 0;
        for (int col = 0; col < nCols; col++) {
            sum_cur_row += a[row][col];
        }
        if (sum_cur_row != target_sum) {
            return false;
        }
    }

    // Check if columns are equal
    for (int col = 0; col < nCols; col++) {
        int sum_cur_col = 0;
        for (int row = 0; row < nRows; row++) {
            sum_cur_col += a[row][col];
        }
        if (sum_cur_col != target_sum) {
            return false;
        }
    }
```

```
    // Calculate first diagonal (Main)
    int sum_left_diagonal = 0;
    for (int i = 0; i < nRows; i++) {
        sum_left_diagonal += a[i][i];
    }
    if (sum_left_diagonal != target_sum) return false;

    // Calculate second diagonal (Anti)
    int sum_right_diagonal = 0;
    for (int i = 0; i < nRows; i++) {
        sum_right_diagonal += a[i][nCols - 1 - i];
    }
    if (sum_right_diagonal != target_sum) return false;

    return true;
}

int main() {
    int nRows, nCols;
    cin >> nRows >> nCols;

    // Allocate memory
    int** arr = new int* [nRows];
    for (int i = 0; i < nRows; i++) {
        arr[i] = new int[nCols];
    }

    for (int i = 0; i < nRows; i++) {
        for (int j = 0; j < nCols; j++) {
            cin >> arr[i][j];
        }
    }

    // Check if magic square
    if (isMagicSquare(arr, nRows, nCols)) {
        cout << "true" << endl;
    }
    else {
        cout << "false" << endl;
    }

    // Deallocate memory
    for (int i = 0; i < nRows; i++) {
        delete[] arr[i];
    }
    delete[] arr;

    return 0;
}
```

3.3 Test cases

Test case 1: $\begin{bmatrix} 2 & 7 & 6 \\ 9 & 5 & 1 \\ 4 & 3 & 8 \end{bmatrix} \rightarrow$ true

Test case 2: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix} \rightarrow$ false

Test case 3: $\begin{bmatrix} 2 & 2 & 2 \\ 1 & 5 & 3 \\ 4 & 1 & 4 \end{bmatrix} \rightarrow$ false

4 Question 4

Question 4

Given a singly linked list defined as follows:

```
struct Node {
    int data;
    Node* pNext;
};

struct List {
    Node* pHead;
};
```

Complete the following function to **remove all nodes** whose values are **coprime** with a given integer x.

```
void removeCoprimeNodes(List& l, int x);
```

Notes: Two integers are **coprime** if their greatest common divisor (GCD) is equal to 1.

Input	Output
List: $[6 \rightarrow 9 \rightarrow 10 \rightarrow 14]$, $x = 7$	$[14]$
List: $[4 \rightarrow 6 \rightarrow 8 \rightarrow 9]$, $x = 3$	$[6 \rightarrow 9]$

4.1 Algorithm

I follow this algorithm:

1. Get the count of numbers the user will input.
2. Let the user input the numbers into the list.
3. Let the user input the checking number (x).
4. Traverse the list to filter nodes:
   (a) While the head node's data is coprime with x:

- Move the `pHead` pointer to the next node.
- Delete the old head node.

(b) Traverse the rest of the list with two pointers (current and previous):

- If the data of the current node is coprime with x, delete the node and link the previous node to the next node.
- Otherwise, move both pointers forward.

4.2 Code

```cpp
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* pNext;
};

struct List {
    Node* pHead;
};

Node* CreateNode(int val) {
    Node* newnode = new Node();
    newnode->data = val;
    newnode->pNext = nullptr;

    return newnode;
}

void AddTail(List& l, int val) {
    Node* newnode = CreateNode(val);

    if (l.pHead == nullptr) {
        l.pHead = newnode;
        return;
    }

    Node* temp = l.pHead;
    while (temp->pNext != nullptr) {
        temp = temp->pNext;
    }
    temp->pNext = newnode;
}

void printList(Node* head) {
    cout << "[";
    if (head != nullptr) {
        Node* cur = head;
        while (cur != nullptr) {
            cout << cur->data;
            if (cur->pNext != nullptr) {
                cout << " -> ";
            }
            cur = cur->pNext;
        }
    }
    cout << "]";
}
```

```
51
52  void deleteList(Node*& head) {
53      Node* cur = head;
54      while (head != nullptr) {
55          head = head->pNext;
56          delete cur;
57          cur = head;
58      }
59  }
60
61  bool isCoprime(int num1, int num2) {
62      // Euclidean Algorithm
63      if (num1 == 0 || num2 == 0) return (num1 == 1 && num2 == 1);
64
65      int a = num1, b = num2;
66      while (b != 0) {
67          int temp = b;
68          b = a % b;
69          a = temp;
70      }
71      return (a == 1);
72  }
73
74  void removeCoprimeNodes(List& l, int x) {
75      // 1. Handle the head(s)
76      while (l.pHead != nullptr && isCoprime(l.pHead->data, x)) {
77          Node* temp = l.pHead;
78          l.pHead = l.pHead->pNext;
79          delete temp;
80      }
81
82      // 2. Handle the body
83      if (l.pHead == nullptr) return;
84
85      Node* prev = l.pHead;
86      Node* cur = l.pHead->pNext;
87
88      while (cur != nullptr) {
89          if (isCoprime(cur->data, x)) {
90              Node* temp = cur;
91              prev->pNext = cur->pNext;
92              cur = cur->pNext;
93              delete temp;
94          }
95          else {
96              prev = cur;
97              cur = cur->pNext;
98          }
99      }
100 }
101
102 int main() {
103     List l;
104     l.pHead = nullptr;
105
106     // Input
107     int n;
108     cout << "Type the count of numbers: ";
109     cin >> n;
110
111     int m;
112     cout << "Type the numbers: ";
```

```
    for (int i = 0; i < n; i++) {
        cin >> m;
        AddTail(l, m);
    }

    int x;
    cout << "Type the check integer: ";
    cin >> x;

    removeCoprimeNodes(l, x);

    printList(l.pHead);

    deleteList(l.pHead);

    return 0;
}
```

4.3 Test cases

Test case 1:
List: $[5 \to 10 \to 15 \to 7]$, $x = 5 \to [5 \to 10 \to 15]$
Test case 2:
List: $[8 \to 12 \to 7 \to 14 \to 21]$, $x = 7 \to [7 \to 14 \to 21]$
Test case 3:
List: $[1 \to 2 \to 3 \to 4]$, $x = 1 \to []$

5 Question 5

Question 5

Given an input file named `input.txt` that contains information of students. Each line has the following format:

```
Name, Math score, Literature score, English score
```

where the fields are separated by commas:

- `Name` is a string (e.g., `Nguyen Van A`).
- Scores are real numbers.

Write a program that:

- Reads data from `input.txt`.
- Computes the GPA (average of the three scores) for each student.
- Writes to `output.txt` the list of students who have the **highest GPA**.

For example

`input.txt`

```
Nguyen Van A,8,7,9
Tran Thi B,9,9,9
Le Van C,9,9,9
```

`output.txt`

```
Tran Thi B,9.0
Le Van C,9.0
```

Note:

- GPA should be printed with one decimal place.
- If multiple students share the highest GPA, print all of them.

5.1 Algorithm

The algorithm employed in the program can be described as follows:

1. **First Pass: Determine the Maximum GPA**
   - Open the input file `input.txt` for reading.
   - Initialize a variable `maxGPA` to -1.0 (a value lower than any possible GPA).
   - For each line in the file:
     - Parse the line using a string stream, extracting:
       - The student's name (text up to the first comma).
       - The Math, Literature, and English scores (as strings, delimited by commas).
     - Convert the score strings to floating-point values.
     - Compute the current student's GPA as the arithmetic mean of the three scores.
     - If this GPA is greater than or equal to `maxGPA`, update `maxGPA` accordingly.
   - Close the input file.
2. **Second Pass: Output Students with the Highest GPA**
   - Re-open `input.txt` for reading.
   - Open `output.txt` for writing.
   - Configure the output stream to display floating-point numbers with exactly one decimal place using fixed-point notation.
   - For each line in the file (parsing identically to the first pass):
     - Compute the student's GPA as before.
     - If the GPA is equal to `maxGPA`, write the student's name followed by a comma and the GPA (formatted to one decimal place) to `output.txt`, followed by a newline.
   - Close both files.

This two-pass approach ensures that all students sharing the highest GPA are correctly identified and listed in the order they appear in the input file, without requiring storage of the entire dataset in memory.

5.2 Code

```cpp
#include <fstream>
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

int main() {
    ifstream inFile("input.txt");

    if (!inFile) {
        cout << "Error opening file!" << endl;
        return 1;
    }
```

```
    double result_score = -1.0;
    string line;
    string name;
    double math_score, literature_score, english_score;

    // Read each line to find Max GPA
    while (getline(inFile, line)) {
        if (line.empty()) continue;

        stringstream ss(line);

        // Read name (until comma)
        getline(ss, name, ',');

        string math_str, lit_str, eng_str;
        getline(ss, math_str, ',');
        getline(ss, lit_str, ',');
        getline(ss, eng_str); // Read until end of line

        try {
            math_score = stod(math_str);
            literature_score = stod(lit_str);
            english_score = stod(eng_str);

            double cur_result = (math_score + literature_score + english_score) / 3.0;

            if (cur_result > result_score) {
                result_score = cur_result;
            }
        } catch (...) {
            continue; // Skip malformed lines
        }
    }
    inFile.close();

    // Second Pass
    ifstream inFile2("input.txt");
    ofstream outFile("output.txt");

    if (!inFile2) {
        return 1;
    }

    outFile << fixed << setprecision(1);

    while (getline(inFile2, line)) {
        if (line.empty()) continue;

        stringstream ss(line);

        getline(ss, name, ',');

        string math_str, lit_str, eng_str;
        getline(ss, math_str, ',');
        getline(ss, lit_str, ',');
        getline(ss, eng_str);

        try {
            math_score = stod(math_str);
            literature_score = stod(lit_str);
            english_score = stod(eng_str);
```

```
78
79              double cur_result = (math_score + literature_score + english_score) / 3.0;
80
81              // Use a small epsilon for float comparison if strictly needed,
82              // but strict equality is acceptable here given identical calculation order.
83              if (cur_result == result_score) {
84                  outFile << name << "," << cur_result << endl;
85              }
86          } catch (...) {
87              continue;
88          }
89      }
90
91      inFile2.close();
92      outFile.close();
93      return 0;
94  }
```

5.3 Test cases

Test case 1:
input.txt

```
Nguyen Thi Lan,10.0,10.0,10.0
Hoang Van Minh,9.5,9.5,9.5
Pham Anh Tu,8.0,8.5,9.0
```

output.txt

```
Nguyen Thi Lan,10.0
```

Test case 2:
input.txt

```
Tran Van Hung,8.5,9.0,8.0
Le Thi Mai,9.0,8.5,9.0
Bui Xuan Hoa,9.0,9.0,8.5
Do Quang Vinh,8.5,8.5,9.0
```

output.txt

```
Le Thi Mai,8.8
Bui Xuan Hoa,8.8
```

Test case 3:
input.txt

```
Vuong Dinh Hue,7.5,7.5,7.5
Nguyen Phu Trong,7.5,7.5,7.5
Pham Minh Chinh,6.0,8.0,7.0
```

output.txt

```
Vuong Dinh Hue,7.5
Nguyen Phu Trong,7.5
```